C Report Part I

Katie Wan, Ashlyn Chor, Suyoung Moon, Jiahui Yan

May-June 2026

1 Group Organisation

1.1 Work Split and Coordination

To ensure equal contribution and enable us all to work in parallel, we divided the emulator's execution pipeline into the different instructions. Each member implemented the decoding and execution of each instruction type of the A64 instruction set:

- **Data Processing (Immediate):** Jiahui
- **Data Processing (Register):** Suyoung
- **Single Data Transfer (Load-Store):** Ashlyn
- **Branching:** Katie

We established internal milestones to maintain momentum, completing the emulator on 28/05 against a planned 27/05 deadline after a few refactoring setbacks.

For version control, we utilise branching with GitLab merge requests to prevent merge conflicts, using conventional naming systems. To maintain project hygiene, we automated our merge checking by integrating a CI/CD pipeline into our `Makefile` system that automatically runs `clang-format`, builds the project, and executes the provided test suite. We follow conventional commits for our commit messages, e.g., feat, chore, etc. ensuring that others can easily understand the purpose of a commit in the master branch logs. Discord allows us to communicate for progress updates and code reviews, where documentation (such as our coding style conventions) is pinned for easy access.

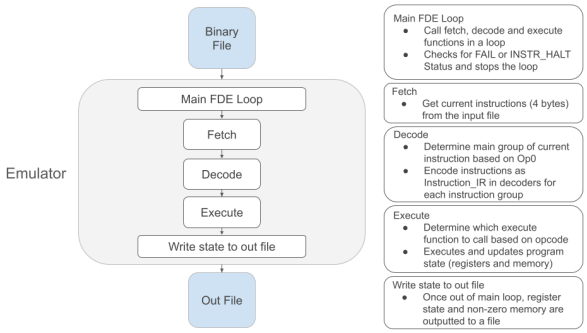
1.2 Group Dynamic and Future Adaptations

Our group dynamic is collaborative. While members initially worked independently on their assigned instruction components, we regularly held team discussions to discuss overarching architectural decisions (e.g., how to implement error checking, or the internal representation of instructions). We communicate our external commitments that may affect the lack of work within that time span, allowing us to rebalance workloads and ensure fairness.

While code reviews were effective for the emulator, the Assembler may require more communication to speed the process up. Consequently, we plan to adapt our workflow by **pair programming**. Having one member write the code while the other reviews in real-time may be beneficial for navigating the logic required for the assembler implementation, as we can catch logic bugs and segmentation faults much earlier, while also working together to reuse what is already implemented by the emulator.

2 Implementation Strategies

2.1 Emulator Structure



2.1.1 FDE cycle

We built an emulator loop that continuously fetches a 32-bit instruction word from memory and passes it to our decode module. To maximise modularity, we split the decoding logic into separate dispatch functions (e.g., `dispatch_dp_immediate`, `dispatch_arithmetic`), using a switch statement to route instructions based on their opcodes. The binary word is translated into our Internal Representation using the `Instruction_IR` struct. This struct utilises a `union` to efficiently store the operands for each instruction group. By doing this, each execution function takes in this parsed struct, abstracting the bitwise decoding from the actual execution logic.

2.1.2 Error Handling

`EXIT_FAILURE` is a macro in C that indicates an unsuccessful termination status, typically used as an argument to the `exit()` function to signal that a program has ended due to an error.

We used an `InstrResult` enum to allow instructions to report their status. If an instruction encounters a fatal error, it will return `INSTR_FAIL` instead of `INSTR_OK`. The fetch-decode-execute loop will then catch this return value and process it accordingly. This method is also used for the `halt` instruction by a special status code `INSTR_HALT`.

2.1.3 Dynamic Memory Allocation

We use `calloc` to dynamically allocate memory on the heap. The specification requires the memory to have a capacity of 2MB (2×2^{20} bytes). By using `calloc`, we avoid stack overflow issues for this large space and naturally fill the initialised memory with 0s.

2.2 Reusability for Assembler

We plan to reuse our error handling enum and altering it to add more specific error values for assembler, such as, `SYMBOL_NOT_FOUND` errors. Also, before generating the binary code, we can reuse the `Instruction_IR` struct to keep our definition of instructions consistent across the emulator codebase.

2.3 Future Implementation Challenges and Solutions

During emulator, we encountered 2 big refactors. Our first major refactor was adding error handling, as this required us update our function definitions to return a status enum. Our second refactor was setting up an internal representation of each instruction, which was specifically done to reuse it when we build the assembler. For the assembler, we plan to plan our implementation beforehand to mitigate the need for large refactors, which both lead to needing to debug our entire codebase.

In the assembler, we will encounter branching instructions, which jump to labels that have not been defined yet. To solve this forward reference challenge, we will implement a Two-Pass assembly process: pass one will build a symbol table mapping labels to memory addresses and the second pass will translate each line of assembly to our IR of an instruction, substitute the addresses of forward references into the instruction and generate the binary.